

Metastability Characterizer for GF180 Flip-Flops

ECE 298A, University of Waterloo · Benjamin Eisenberg and Roshan Hegde · Tiny Tapeout, GF180MCU

Status: design in progress; target shuttle submission 7 December 2026.

Purpose

This project aims to improve the speed and reliability of data transfer between digital circuits operating on independent clocks. When a signal changes close to the clock edge of the flip-flop that receives it, the flip-flop can enter a metastable state: its output lingers between logic 0 and logic 1 and resolves only after an unbounded, probabilistic delay [1]. The probability that the output remains unresolved after a time t decays as $e^{-t/\tau}$, where the resolution time constant τ is set by the regeneration of the flip-flop’s internal latch [1, 5]. Because τ appears in an exponent, a modest reduction in τ produces a large reduction in failure rate. Measured values of τ in modern processes have been worse than device scaling would suggest [2], and the circuit design of the flip-flop is one of the principal factors that set it [4]. Prior work has analysed how transistor mismatch and unequal loading affect the metastable behaviour of CMOS latches [9], and has proposed latch circuits that accelerate resolution [10, 11].

The chip measures τ for four flip-flops fabricated side by side under identical conditions: the GF180 library cell `dffq_1`, a baseline flip-flop built from standard logic cells, a design incorporating one targeted modification, and a control variant. In ngspice simulations using the GF180MCU process models [6, 7], the modified design’s resolution time constant is approximately 11% shorter than the baseline’s at typical conditions (57.5 ps against 64.5 ps). The purpose of the chip is to test this prediction in silicon.

How it works

The chip implements the on-chip early/late sampling method of Beer *et al.* [3]. A sequencer driven by the 20 MHz system clock runs one trial every 200 ns, or five million trials per second.

- **Stimulus.** A 31-stage ring oscillator, divided by four, produces data edges that are asynchronous to the system clock. An external data pin can be selected instead.
- **Capture.** All four flip-flops capture the same data on a shared capture clock. A 4:1 multiplexer selects one flip-flop for observation.
- **Early sample.** The selected output is sampled after a programmable delay d_k , chosen from 16 settings spaced approximately 45 ps apart.
- **Late samples.** The output is sampled again at 50 ns and 100 ns, when it has settled. A disagreement between the early and late samples indicates that the flip-flop had not resolved by d_k .
- **Counting and calibration.** Disagreements K are counted over blocks of $N = 255$ trials and held on the output pins for readout; a calibration mode measures the actual delay of each setting on the chip.

Off chip, the disagreement rate $\hat{p}_k = K/N$ is computed for each setting, $\ln \hat{p}_k$ is fitted against the calibrated delay d_k , and $\tau = -1/\text{slope}$. Because all four flip-flops share one delay line and one calibration, ratios of τ between flip-flops are insensitive to errors in the delay scale.

Pinout (draft)

Signal	Dir.	Pins
Clock, active-low reset	In	<code>clk</code> , <code>rst_n</code>
Flip-flop select [1:0]	In	<code>ui</code> [3], <code>uio</code> [6] (bit 1)
Delay code [3:0]	In	<code>ui</code> [2:0], <code>uio</code> [7] (bit 3)
Start, source, ext. data, mode	In	<code>ui</code> [4]– <code>ui</code> [7]
Ring oscillator enable	In	<code>uio</code> [0]
Held 8-bit count	Out	<code>uo</code> [7:0]
Busy, done, fault	Out	<code>uio</code> [1]– <code>uio</code> [3]
Source and trial monitors	Out	<code>uio</code> [4], <code>uio</code> [5]

Key parameters: GF180MCU standard cells [6]; 3.3 V supply (to be confirmed); one Tiny Tapeout tile, about 1,900 transistors (estimate); 20 MHz clock; 5×10^6 trials/s; 16 delay settings; $\pm 3\%$ target on τ ratios.

Team: Benjamin Eisenberg (flip-flop design, circuit simulation, delay line, analysis) and Roshan Hegde (sequencer, counters, readout, verification); integration, layout checks and testing are shared.

References

- [1] R. Ginosar, “Metastability and synchronizers: A tutorial,” *IEEE Design & Test of Computers*, vol. 28, no. 5, pp. 23–35, 2011.
- [2] S. Beer, R. Ginosar, M. Priel, R. Dobkin, and A. Kolodny, “The devolution of synchronizers,” in *Proc. IEEE ASYNC*, 2010, pp. 94–103, doi: 10.1109/ASYNC.2010.22.
- [3] S. Beer, R. Ginosar, M. Priel, R. Dobkin, and A. Kolodny, “An on-chip metastability measurement circuit to characterize synchronization behavior in 65nm,” in *Proc. IEEE ISCAS*, 2011, pp. 2593–2596, doi: 10.1109/ISCAS.2011.5938135.
- [4] S. Beer and R. Ginosar, “Eleven ways to boost your synchronizer,” *IEEE Trans. VLSI Systems*, vol. 23, no. 6, pp. 1040–1049, 2015, doi: 10.1109/TVLSI.2014.2331331.
- [5] C. L. Portmann, “Characterization and reduction of metastability errors in CMOS interface circuits,” Ph.D. dissertation, Stanford University, 1995, Tech. Rep. CSL-TR-95-671.
- [6] GF180MCU open-source PDK and `gf180mcu_fd_sc_mcu7t5v0` standard-cell library, <https://gf180mcu-pdk.readthedocs.io>.
- [7] ngspice, open-source mixed-signal circuit simulator, version 42, <https://ngspice.sourceforge.io>.
- [8] Tiny Tapeout documentation: tile pinout and demo board, <https://tinytapeout.com>.
- [9] K. O. Jeppson, “Comments on the metastable behavior of mismatched CMOS latches,” *IEEE J. Solid-State Circuits*, vol. 31, no. 2, pp. 275–277, 1996.
- [10] J. Zhou, D. Kinniment, G. Russell, and A. Yakovlev, “A robust synchronizer,” in *Proc. IEEE ISVLSI*, 2006, pp. 442–443, doi: 10.1109/ISVLSI.2006.12.
- [11] I. W. Jones, S. Yang, M. R. Greenstreet, H. N. Gaywala, and R. J. Drost, “Synchronizer latch circuit that facilitates resolving metastability,” U.S. Patent 8,552,779 B2 (Oracle), Oct. 8, 2013.

How to test

The Tiny Tapeout demo board supplies the 20 MHz clock, drives the inputs and reads the outputs [8].

1. Hold `rst_n` low, start the clock, release reset.
2. In calibration mode, step through the 16 delay codes; for each, assert START, wait for DONE, record the count.
3. In measurement mode, enable the ring oscillator and select it as the source. For each flip-flop and delay code, assert START, wait for DONE, read `uo`[7:0]; discard any block that raises FAULT.
4. Fit $\ln \hat{p}_k$ against the calibrated delays to obtain τ .

An oscilloscope on `uio`[4] and `uio`[5] confirms that the source toggles and that trials occur every 200 ns.